

Resolución del Set Covering Problem (SCP) mediante Simulated Annealing

Juan Geraldo - Eva Ponce - Eduardo Sandoval

18 de junio de 2026

Resumen

Este informe documenta la implementación de la metaheurística de **Simulated Annealing** para resolver el *Set Covering Problem* (SCP), siguiendo la heurística de búsqueda local de Jacobs & Brusco (1995). El algoritmo opera sobre una representación binaria de la solución y un vecindario de *remoción y reparación* parametrizado por ρ_1 y ρ_2 . Se evalúan tres instancias de complejidad creciente ($m \in \{500, 1000, 2000\}$) con un presupuesto de 1000 evaluaciones (o 60 s) por corrida y 10 réplicas independientes y reproducibles por instancia. Los resultados muestran que el método converge de forma **perfectamente estable** (desviación estándar nula en las tres instancias) a soluciones de costo 5, 433 y 269, respectivamente.

1. Set Covering Problem (SCP)

El Set Covering Problem (SCP) es un problema clásico de optimización combinatoria. El problema se define sobre un conjunto universo de n elementos, denotado como U , y una colección de m subconjuntos, denominados S_i con $i \in \{1, 2, \dots, m\}$. Cada subconjunto tiene asociado un costo particular. El objetivo fundamental radica en seleccionar una combinación de estos subconjuntos tal que se logre cubrir la totalidad de los elementos del universo U , minimizando simultáneamente el costo total de los subconjuntos seleccionados.

1.1. Análisis del Modelo Matemático

El modelo matemático validado se compone de:

Parámetros:

- n : Representa el número total de elementos.
- m : Número total de subconjuntos.
- c_i : Costo asociado al subconjunto S_i .
- a_{ji} : Parámetro binario que toma el valor de 1 si el elemento j pertenece al subconjunto i , y 0 en caso contrario.

Variables de Decisión: Se define x_i como una variable binaria donde $x_i = 1$ si el subconjunto S_i es seleccionado para formar parte de la solución, y $x_i = 0$ en caso contrario. El dominio está restringido a:

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, m\} \quad (1)$$

Función Objetivo:

$$\text{mín } z = \sum_{i=1}^m c_i x_i \quad (2)$$

Restricciones de Cobertura: La restricción de cobertura asegura matemáticamente que cada objeto del universo U se encuentre, como mínimo, en un subconjunto de los que conforman la solución final:

$$\sum_{i=1}^m a_{ji} x_i \geq 1, \quad \forall j \in \{1, 2, \dots, n\} \quad (3)$$

1.2. Estructura de la Instancia

Las instancias de prueba utilizadas frecuentemente en la literatura (como las del repositorio OR-Library) poseen una estructura de texto plano organizada en bloques secuenciales:

1. **Dimensiones:** La primera línea del archivo contiene dos valores enteros, indicando el número total de elementos n y el número total de subconjuntos m .
2. **Vector de Costos:** Una secuencia de m valores numéricos que representan los costos c_i asociados a cada subconjunto S_i .
3. **Matriz de Cobertura (Representación Dispersa):** La última sección detalla la cobertura elemento por elemento. Para cada elemento j , se indica primero la cantidad de subconjuntos que lo cubren, seguido de los identificadores exactos de esos subconjuntos.

1.3. Análisis Inicial de Variables y Restricciones

La abstracción de estos datos hacia el dominio del problema define el espacio de búsqueda del algoritmo a diseñar:

- **Espacio de Soluciones:** El problema consiste en determinar un vector binario de tamaño m . Dado que cada variable x_i tiene un dominio discreto $\{0, 1\}$, el espacio total de combinaciones posibles es de 2^m . Para una instancia con $m = 1000$, un enfoque de fuerza bruta debería recorrer teóricamente 2^{1000} estados, lo cual exige una acotación inteligente de la región factible mediante una metaheurística.
- **Restricciones y Factibilidad:** Cualquier vector binario candidato debe someterse a la validación de la matriz dispersa. Si la sumatoria cruzada de la ecuación (3) para algún elemento j resulta ser 0, la solución entera se declara infactible y se descarta.

2. Descripción General

2.1. Definición y clasificación

Simulated Annealing es una metaheurística de **trayectoria estocástica** (*single-solution based metaheuristic*), la cual mantiene y refina una *única* solución incumbente a lo largo del tiempo, en contraste con las metaheurísticas poblacionales (como los algoritmos genéticos), que manipulan un conjunto de soluciones de forma simultánea. El método se inspira en el proceso metalúrgico de recocido, en el que un material se calienta y luego se enfría de forma controlada para alcanzar un estado de mínima energía.

Conceptualmente, SA es una **búsqueda local extendida**, debido a que en cada iteración genera una solución vecina y decide si reemplaza a la actual. La diferencia esencial respecto del ascenso de colina (*hill climbing*) es que SA acepta *deliberadamente* soluciones peores con una probabilidad controlada, gobernada por el **criterio de Metropolis**:

$$P(\text{aceptar } S') = \begin{cases} 1, & \text{si } \delta \leq 0, \\ e^{-\delta/T}, & \text{si } \delta > 0, \end{cases} \quad \delta = z(S') - z(S), \quad (4)$$

donde T es la *temperatura* de control. Esta aceptación probabilística de movimientos de empeoramiento es lo que permite al algoritmo escapar de óptimos locales. La temperatura comienza alta (búsqueda exploratoria, muchos movimientos peores aceptados) y se reduce progresivamente (búsqueda explotadora, casi solo se aceptan mejoras), realizando un tránsito gradual de la *exploración* a la *intensificación*.

2.2. Ventajas y desventajas

Ventajas.

- **Capacidad de escape de óptimos locales:** a diferencia de la búsqueda local pura, el criterio de Metropolis evita el estancamiento prematuro en mínimos locales.
- **Simplicidad y bajo costo de memoria:** al operar sobre una sola solución, la huella de memoria es mínima y la implementación es directa.
- **Generalidad:** solo requiere definir una representación, un vecindario y una función objetivo, lo que lo hace aplicable a una amplia gama de problemas combinatorios.
- **Convergencia asintótica:** bajo esquemas de enfriamiento suficientemente lentos existen garantías teóricas de convergencia al óptimo global.

Desventajas.

- **Sensibilidad al ajuste de parámetros:** el desempeño depende fuertemente de la temperatura inicial, el factor de enfriamiento y el diseño del vecindario.
- **Costo de la convergencia teórica:** las garantías formales exigen enfriamientos tan lentos que resultan muy complejos, por lo cual se usan esquemas aproximados.
- **Ausencia de diversidad poblacional:** al explorar una sola trayectoria, no aprovecha el paralelismo informativo de los métodos poblacionales.
- **Variabilidad estocástica:** en general distintas semillas producen resultados diferentes (aunque, como se verá, en estas instancias la variabilidad fue nula).

2.3. Justificación de la elección para el SCP

La elección de SA para el SCP se sustenta tanto en argumentos algorítmicos como matemáticos:

1. **Naturaleza del espacio de búsqueda.** El SCP es NP-Hard y su espacio de soluciones $\{0, 1\}^m$ crece exponencialmente (2^m). Para la instancia más grande ($m = 2000$) la enumeración exacta es inviable, mientras que SA navega el espacio de forma eficiente sin requerir solvers comerciales.
2. **Compatibilidad con la topología del problema.** La representación binaria del vector de decisión x se corresponde de manera natural con el estado que manipula SA, y el vecindario de remoción-reparación (ρ_1, ρ_2) explota directamente la estructura de cobertura del SCP.
3. **Manejo de la factibilidad.** El operador de reparación golosa garantiza que toda solución vecina vuelva a ser factible (cubre todos los elementos), por lo que la búsqueda se restringe siempre a la región factible y la función objetivo es directamente comparable.
4. **Respaldo en la literatura.** La heurística de búsqueda local de Jacobs & Brusco (1995) fue diseñada específicamente para instancias grandes del SCP, lo que la convierte en una base sólida y validada para esta tarea.

3. Implementación

La metaheurística se implementó íntegramente en Python (solo biblioteca estándar: `random`, `math`, `time`, `statistics`), reutilizando las estructuras de datos binarias de los talleres previos. La arquitectura sigue fielmente el esquema CONSTRUCT + SEARCH + ciclo de Simulated Annealing de Jacobs & Brusco (1995).

3.1. Componentes de la arquitectura

Representación de la solución.

Una solución es un **vector binario** x de longitud m , donde $x_{i-1} = 1$ si y solo si el subconjunto S_i está seleccionado. Para operar de forma eficiente, el algoritmo mantiene además un **arreglo de cobertura** w , donde $w[j]$ es el número de subconjuntos seleccionados que cubren al elemento j . La solución es factible si y solo si $w[j] \geq 1$ para todo $j \in \{1, \dots, n\}$. Las operaciones básicas AGREGAR(i) y QUITAR(i) activan o desactivan un subconjunto y actualizan w de forma *incremental* (solo sobre los elementos que cubre S_i), evitando recalcular la factibilidad completa en cada movimiento.

Solución inicial (módulo CONSTRUCT).

Se construye una solución factible mediante una heurística golosa: se selecciona repetidamente un elemento aún sin cubrir (al azar) y se agrega el *mejor* subconjunto que lo cubre según el **orden natural** (costo ascendente y, ante empate, mayor cobertura primero), hasta cubrir el universo completo. Finalmente se eliminan los subconjuntos redundantes.

Estructura de vecindad (movimiento, módulo SEARCH).

El vecino S' se genera con un esquema de **remoción y reparación** parametrizado:

- ρ_1 controla la *magnitud de la perturbación*: se retiran al azar $D = \lceil \rho_1 \cdot N(S) \rceil$ subconjuntos, acotado a $1 \leq D \leq N(S)$, donde $N(S)$ es el número de subconjuntos seleccionados.
- ρ_2 controla la *profundidad de la reparación*: solo pueden reingresar subconjuntos de costo $c_i \leq E$, con $E = \max(\lceil \rho_2 \cdot Q(S) \rceil, Q(S))$ y $Q(S)$ el costo máximo entre los subconjuntos seleccionados. La cota $E \geq Q(S)$ garantiza la *reparabilidad*: todo subconjunto recién retirado sigue siendo candidato.

La reparación es golosa: mientras existan elementos sin cubrir, se incorpora el subconjunto candidato de menor razón $\beta_i = c_i/\alpha_i$, donde α_i es el número de elementos aún sin cubrir que cubre S_i (los empates se resuelven al azar). Se finaliza eliminando redundancias.

Evaluación de la función objetivo.

El costo de una solución es la suma directa de los costos de los subconjuntos activos, $z(x) = \sum_{i: x_i=1} c_i$, conforme a la ecuación (2).

Esquema de enfriamiento.

Se emplea un enfriamiento **geométrico**: la temperatura se actualiza como $T \leftarrow T \cdot CF$ cada TL iteraciones internas. Junto con el criterio de Metropolis de la ecuación (4), esto define la disminución progresiva de la probabilidad de aceptar empeoramientos.

Seguimiento del incumbente.

El algoritmo distingue entre la solución *actual* (x, z) (que puede empeorar por Metropolis) y la *mejor solución encontrada* (x^*, z^*) , que solo se actualiza ante una mejora estricta. De este modo, aceptar un vecino peor nunca implica perder la mejor solución hallada.

3.2. Pseudocódigo

A continuación se presenta el pseudocódigo de los tres componentes principales, extraído directamente del código implementado.

Algorithm 1 CONSTRUIR — solución inicial factible (golosa)

```

1: function CONSTRUIR(instancia)
2:    $x \leftarrow$  vector de  $m$  ceros;  $w[j] \leftarrow 0 \ \forall j$ 
3:    $sinCubrir \leftarrow \{1, 2, \dots, n\}$ 
4:   while  $sinCubrir \neq \emptyset$  do
5:      $j \leftarrow$  elemento aleatorio de  $sinCubrir$ 
6:      $i \leftarrow$  primer subconjunto que cubre  $j$  según el orden natural
7:     AGREGAR( $i, x, w$ )
8:      $sinCubrir \leftarrow sinCubrir \setminus cubre(i)$ 
9:   end while
10:  ELIMINARREDUNDANTES( $x, w$ )
11:  return  $(x, w)$ 
12: end function

```

Algorithm 2 GENERARVECINO — vecindario de remoción y reparación (ρ_1, ρ_2)

```
1: function GENERARVECINO( $x, w, instancia, \rho_1, \rho_2$ )
2:    $S \leftarrow \{i : x_i = 1\}$ ;  $N \leftarrow |S|$ ;  $Q \leftarrow \max_{i \in S} c_i$ 
3:    $D \leftarrow \min(\max(\lceil \rho_1 \cdot N \rceil, 1), N)$  ▷ cuántos quitar
4:    $E \leftarrow \max(\lceil \rho_2 \cdot Q \rceil, Q)$  ▷ cota de costo para reingresar
5:   retirar al azar  $D$  subconjuntos de  $S$  mediante QUITAR( $\cdot, x, w$ )
6:    $sinCubrir \leftarrow \{j : w[j] = 0\}$ 
7:   while  $sinCubrir \neq \emptyset$  do
8:      $cand \leftarrow \{i : x_i = 0, c_i \leq E, i \text{ cubre algún } j \in sinCubrir\}$ 
9:     for all  $i \in cand$  do
10:       $\alpha_i \leftarrow |\text{cubre}(i) \cap sinCubrir|$ ;  $\beta_i \leftarrow c_i / \alpha_i$ 
11:    end for
12:     $i^* \leftarrow \arg \min_i \beta_i$  ▷ empate al azar
13:    AGREGAR( $i^*, x, w$ )
14:     $sinCubrir \leftarrow sinCubrir \setminus \text{cubre}(i^*)$ 
15:  end while
16:  ELIMINARREDUNDANTES( $x, w$ )
17:  return ( $x, w$ )
18: end function
```

Algorithm 3 RECOCIDOSIMULADO — ciclo principal

```
1: function RECOCIDOSIMULADO( $instancia, \rho_1, \rho_2, T_0, CF, TL, maxEval, maxT$ )
2:    $(x, w) \leftarrow \text{CONSTRUIR}(instancia)$ ;  $z \leftarrow z(x)$ 
3:    $(x^*, z^*) \leftarrow (x, z)$  ▷ mejor solución encontrada
4:    $T \leftarrow T_0$ ;  $evals \leftarrow 0$ ;  $t_0 \leftarrow \text{tiempo actual}$ 
5:   while  $evals < maxEval$  y  $(\text{tiempo} - t_0) < maxT$  do
6:     for TL veces do
7:       if  $evals \geq maxEval$  o se agotó el tiempo then
8:         break
9:       end if
10:       $(x', w') \leftarrow \text{GENERARVECINO}(x, w, instancia, \rho_1, \rho_2)$ 
11:       $z' \leftarrow z(x')$ ;  $evals \leftarrow evals + 1$ ;  $\delta \leftarrow z' - z$ 
12:      if  $\delta \leq 0$  then
13:         $(x, w, z) \leftarrow (x', w', z')$ 
14:        if  $z < z^*$  then
15:           $(x^*, z^*) \leftarrow (x, z)$  ▷ mejora estricta
16:        end if
17:      else if  $\text{rand}() < e^{-\delta/T}$  then
18:         $(x, w, z) \leftarrow (x', w', z')$  ▷ criterio de Metropolis
19:      end if
20:    end for
21:     $T \leftarrow T \cdot CF$  ▷ enfriamiento
22:  end while
23:  return VERIFICAR( $x^*$ )
24: end function
```

El criterio de aceptación de Metropolis se implementa directamente como se muestra

a continuación:

Listing 1: Núcleo del criterio de Metropolis y enfriamiento.

```

1 delta = z_vecino - z
2 if delta <= 0:
3     vector, w, z = vector_vecino, w_vecino, z_vecino
4     if z < mejor_z:
5         mejor_vector = list(vector)
6         mejor_z = z
7 else:
8     if rng.random() < math.exp(-delta / T):
9         vector, w, z = vector_vecino, w_vecino, z_vecino
10 # ... al terminar las TL iteraciones internas:
11 T *= CF # enfriamiento geometrico

```

3.3. Parámetros y recursos computacionales

Los valores de los parámetros se tomaron directamente del código (referencia: Jacobs & Brusco, 1995, Tabla 2) y se resumen en la Tabla 1. Dado que el método es una metaheurística de *trayectoria* (una sola solución), **no** existen parámetros poblacionales (tamaño de población, operador de cruzamiento ni probabilidad de mutación), por lo que se omiten.

Cuadro 1: Parámetros de la metaheurística y presupuesto computacional.

Parámetro	Valor	Rol
ρ_1	0,4	Fracción de subconjuntos a retirar (magnitud del vecindario)
ρ_2	2,0	Cota de costo para reingreso (profundidad de búsqueda)
T_0	1,3	Temperatura inicial
CF	0,9	Factor de enfriamiento geométrico
TL	100	Iteraciones por nivel de temperatura
máx. evaluaciones	1000	Criterio de parada (presupuesto)
tope de tiempo	60 s	Criterio de parada alternativo
corridas	10	Réplicas independientes por instancia
semilla base	12345	La corrida c usa la semilla $12345 + c$

Recursos computacionales asignados. El **presupuesto de cómputo** de cada corrida está acotado por **1000 evaluaciones de la función objetivo** o un tope de **60 segundos** de tiempo de pared, lo que ocurra primero. El bucle interno ejecuta $TL = 100$ iteraciones por nivel de temperatura, de modo que se alcanzan a lo sumo $\lceil 1000/100 \rceil = 10$ niveles de temperatura antes de agotar el presupuesto. La reproducibilidad se garantiza mediante un generador pseudoaleatorio sembrado de forma independiente por corrida (`random.Random(12345 + c)`).

4. Mejor Solución

La Tabla 2 reporta la función objetivo de la mejor solución global $Z(S^*)$ encontrada para cada instancia (sobre las 10 corridas), junto con el número de subconjuntos que la componen. Todas las soluciones son factibles (cubren la totalidad del universo).

Cuadro 2: Mejor solución encontrada por instancia.

Instancia	n	m	$Z(S^*)$
Instancia 1 (01_facil)	50	500	5
Instancia 2 (02_medio)	200	1000	433
Instancia 3 (03_dificil)	200	2000	269

El detalle de los subconjuntos seleccionados (base 1) en cada mejor solución es el siguiente:

- **Instancia 1** ($Z(S^*) = 5$, 5 subconjuntos): $\{1, 5, 21, 113, 237\}$.
- **Instancia 2** ($Z(S^*) = 433$, 71 subconjuntos): $\{1, 2, 3, 5, 6, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 25, 26, 28, 29, 32, 43, 44, 45, 46, 47, 48, 49, 50, 52, 54, 57, 59, 60, 61, 62, 63, 66, 69, 70, 73, 75, 77, 78, 81, 83, 85, 86, 89, 90, 91, 94, 103, 106, 107, 115, 116, 120, 121, 124, 128, 138, 143, 144, 194, 275, 340\}$.
- **Instancia 3** ($Z(S^*) = 269$, 65 subconjuntos): $\{2, 3, 6, 7, 8, 9, 12, 13, 15, 16, 19, 20, 21, 22, 23, 27, 29, 30, 31, 34, 35, 37, 40, 41, 42, 43, 44, 45, 46, 49, 50, 53, 59, 63, 64, 65, 66, 70, 71, 72, 73, 74, 75, 76, 79, 84, 85, 86, 87, 90, 101, 106, 108, 109, 112, 114, 128, 136, 140, 174, 176, 200, 236, 256, 276\}$.

5. Resultados

La Tabla 3 presenta la función objetivo obtenida en cada una de las 10 corridas independientes para las tres instancias, junto con la media y la desviación estándar muestral ($n - 1$). La Figura ?? complementa el análisis con diagramas de cajas comparativos.

Análisis de la estabilidad. El resultado más destacable es la **estabilidad perfecta** del método: las 10 corridas de cada instancia convergen exactamente al mismo valor, produciendo una desviación estándar nula. Esto es notable porque la *construcción inicial sí es estocástica*: según la telemetría del código, el costo de la solución inicial varía entre 7 y 8 (Instancia 1), entre 448 y 453 (Instancia 2) y entre 279 y 283 (Instancia 3). Es decir, partiendo de soluciones iniciales *distintas*, el algoritmo converge invariablemente al mismo óptimo. Asimismo, el criterio de Metropolis está *activo*: se aceptaron 165, 718 y 417 movimientos de empeoramiento en total (sumando las 10 corridas) para las instancias 1, 2 y 3 respectivamente, lo que confirma que la búsqueda no es un mero descenso voraz, sino que explora el espacio antes de asentarse.

La contrapartida de esta estabilidad es que el diagrama de cajas pierde su expresividad habitual (no hay dispersión que mostrar). Por ello, la verdadera evidencia del funcionamiento del método son las trayectorias de convergencia y la distribución de costos iniciales frente al óptimo alcanzado; estas pueden incluirse como figura complementaria.

Cuadro 3: Función objetivo de las 10 corridas por instancia, media y desviación estándar.

Corrida	Instancia 1 ($n=50, m=500$)	Instancia 2 ($n=200, m=1000$)	Instancia 3 ($n=200, m=2000$)
1	5	433	269
2	5	433	269
3	5	433	269
4	5	433	269
5	5	433	269
6	5	433	269
7	5	433	269
8	5	433	269
9	5	433	269
10	5	433	269
Media	5,0000	433,0000	269,0000
Desv. estándar	0,0000	0,0000	0,0000
Mínimo (mejor)	5	433	269
Máximo (peor)	5	433	269

5.1. Convergencia y dinámica de la búsqueda

Para evidenciar que la estabilidad observada no implica un comportamiento trivial, las Figuras 1, 2 y 3 presentan un diagnóstico de la búsqueda para cada instancia. Cada diagnóstico se compone de tres vistas: **(a)** la *convergencia* del mejor costo z^* por evaluación (una línea por corrida); **(b)** la *trayectoria* de una corrida representativa, que contrasta el costo actual aceptado con el mejor incumbente; y **(c)** la *distribución* del costo de la construcción golosa sobre 300 semillas frente al óptimo alcanzado por el SA (línea discontinua).

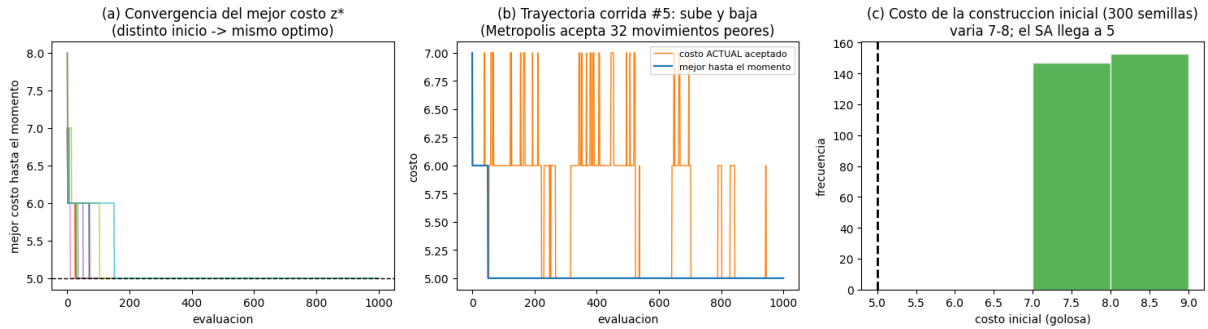


Figura 1: Diagnóstico de la búsqueda — Instancia 1 (fácil, $n=50, m=500$). (a) Las 10 corridas convergen al mismo óptimo $z^* = 5$ desde inicios distintos. (b) Trayectoria de una corrida representativa: Metropolis acepta 32 empeoramientos. (c) Costo de la construcción golosa (300 semillas, rango 7–8) frente al óptimo del SA.

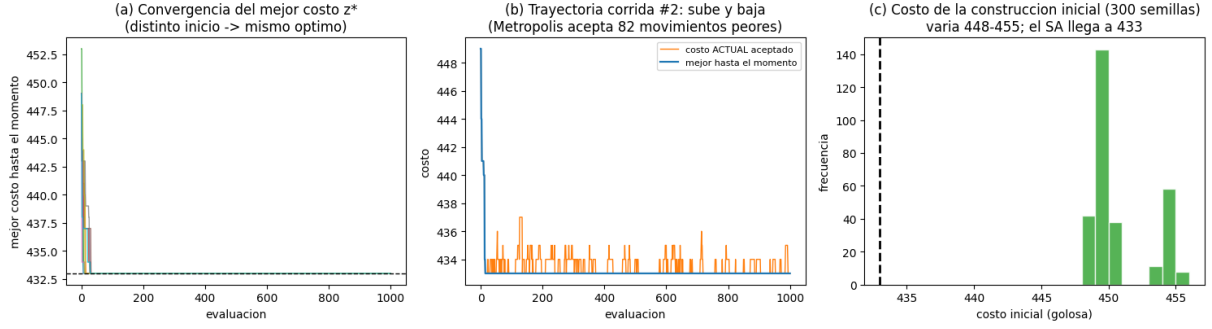


Figura 2: Diagnóstico de la búsqueda — Instancia 2 (media, $n=200$, $m=1000$). (a) Convergencia al óptimo $z^* = 433$. (b) Trayectoria de una corrida representativa: Metropolis acepta 82 empeoramientos. (c) Costo de la construcción golosa (300 semillas, rango 448–455) frente al óptimo del SA.

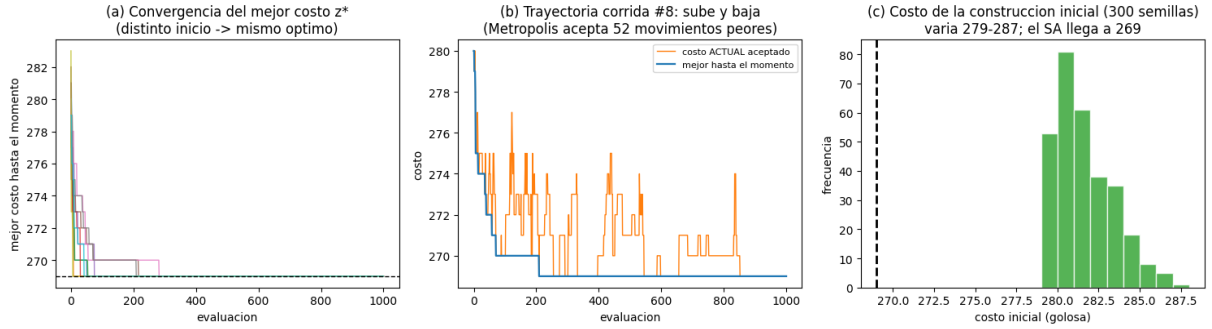


Figura 3: Diagnóstico de la búsqueda — Instancia 3 (difícil, $n=200$, $m=2000$). (a) Convergencia al óptimo $z^* = 269$. (b) Trayectoria de una corrida representativa: Metropolis acepta 52 empeoramientos. (c) Costo de la construcción golosa (300 semillas, rango 279–287) frente al óptimo del SA.

Lectura de los diagnósticos. El panel (a) de las tres instancias confirma el origen de la desviación estándar nula: aunque las corridas parten de soluciones iniciales distintas, todas las trayectorias del mejor costo descienden y se estabilizan en el mismo valor ($z^* = 5$, 433 y 269). Es relevante observar que la convergencia ocurre de forma *temprana* —dentro de las primeras 200 a 300 evaluaciones— y que el resto del presupuesto (hasta 1000) transcurre sin mejoras; esto indica que el presupuesto asignado es holgado para estas instancias y respalda la propuesta de esquemas *multi-start* discutida en las conclusiones.

El panel (b) es la evidencia visual más directa del criterio de Metropolis: la curva del costo *actual aceptado* (naranja) supera repetidamente al mejor incumbente (azul); es decir, el algoritmo acepta de forma deliberada soluciones peores para escapar de óptimos locales. El número de empeoramientos aceptados en la corrida mostrada (32, 82 y 52 para las instancias fácil, media y difícil) confirma que la búsqueda no es un mero descenso voraz.

El panel (c) muestra que la heurística golosa, por sí sola, nunca alcanza el óptimo: su costo se distribuye en rangos de 7–8 (fácil), 448–455 (media) y 279–287 (difícil), siempre

por encima del valor que logra el SA (línea discontinua). Esto cuantifica la mejora que aporta la fase de Simulated Annealing sobre la solución inicial.

5.2. Comparación del proceso entre instancias

Puesto que la función objetivo final es invariante, una comparación informativa debe centrarse en el *proceso* de búsqueda y no en su resultado. La Figura 4 compara las tres instancias en tres métricas de proceso derivadas de la telemetría.

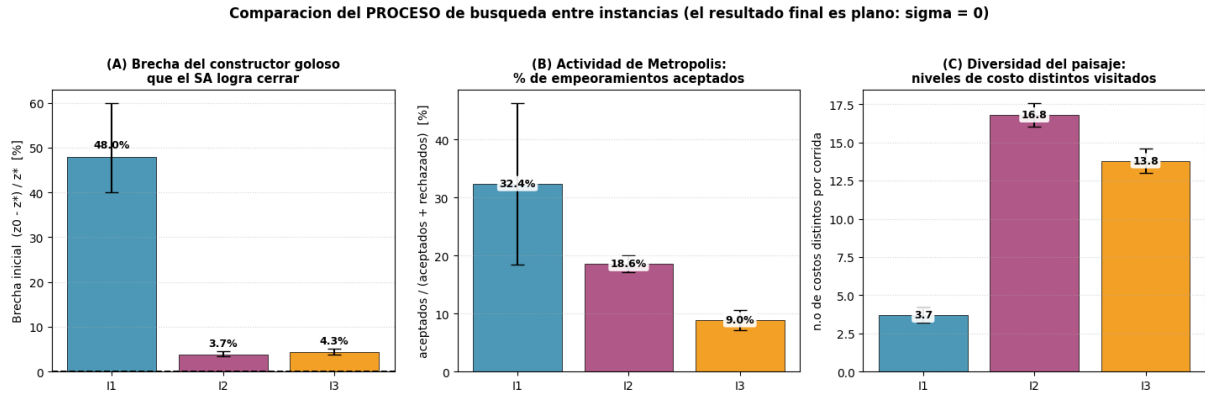


Figura 4: Comparación del *proceso* de búsqueda entre instancias. Como la función objetivo final es invariante ($\sigma = 0$), se comparan métricas de proceso: (A) brecha relativa del constructor goloso que el SA cierra por completo; (B) tasa de aceptación de empeoramientos (criterio de Metropolis), que decae al crecer la instancia; (C) niveles de costo distintos visitados por corrida.

Análisis comparativo. El panel (A) expresa la brecha relativa del constructor goloso, $(z_0 - z^*)/z^*$, que el SA cierra por completo. Esta brecha es grande en la instancia fácil (48 %) pero modesta en las instancias media y difícil (3,7 % y 4,3 %): en las instancias grandes la construcción golosa ya entrega soluciones cercanas al óptimo, de modo que el rol del SA se desplaza desde una mejora sustancial hacia un *pulido fino* y la confirmación del óptimo.

El panel (B) reporta la tasa de aceptación de empeoramientos, $\frac{\text{aceptados}}{\text{aceptados} + \text{rechazados}}$, que decrece de forma monótona al crecer la instancia (32,4 % $\rightarrow$ 18,6 % $\rightarrow$ 9,0 %). La explicación es directa a partir de la ecuación (4): en instancias mayores los incrementos de costo δ son más grandes en relación con la temperatura T , por lo que un menor número de empeoramientos supera el umbral $e^{-\delta/T}$. En consecuencia, el algoritmo transita de un régimen exploratorio (instancia fácil) a uno de descenso más guiado (instancia difícil).

El panel (C) cuantifica la diversidad del paisaje explorado mediante el número de niveles de costo distintos visitados por corrida. Nótese que esta diversidad *no* es monótona en el tamaño m : la instancia media (16,8) explora más niveles que la difícil (13,8). Esto sugiere que la rugosidad del espacio de búsqueda depende de la estructura de costos particular de cada instancia y no solo de su dimensión.

En síntesis, la estabilidad perfecta ($\sigma = 0$) no refleja inactividad del algoritmo sino *robustez*: el SA explora activamente —acepta empeoramientos y recorre múltiples niveles de costo— y, partiendo de orígenes distintos, converge de forma fiable a la misma cuenca de atracción dentro del presupuesto fijado.

6. Conclusiones

¿Qué se aprendió a nivel algorítmico? La implementación evidenció que, en una metaheurística de trayectoria, el **diseño del vecindario es tan determinante como el mecanismo de aceptación**. El vecindario de remoción-reparación (ρ_1, ρ_2) , guiado por la razón costo/cobertura β_i , resulta mucho más efectivo que un *bit-flip* ingenuo, porque cada movimiento conduce siempre a una solución factible y de buena calidad. También se comprendió en la práctica el rol del criterio de Metropolis (la telemetría muestra cientos de empeoramientos aceptados que permiten salir de óptimos locales) y la importancia de separar la solución *actual* de la *mejor encontrada* para no perder calidad al explorar. Finalmente, la eliminación de redundancias se reveló como un paso de intensificación barato pero decisivo para reducir el costo.

¿Cuáles fueron los desafíos de implementación más críticos? El desafío central fue **mantener la factibilidad y la eficiencia de forma simultánea**. Recalcular la cobertura completa en cada movimiento sería $O(n \cdot m)$; en su lugar, el arreglo w se actualiza de forma incremental en $O(|\text{cubre}(i)|)$ por inserción o eliminación. El segundo desafío fue **garantizar la reparabilidad** del vecino: si la cota de reingreso E fuese demasiado restrictiva, la reparación podría quedar sin candidatos y producir una solución infactible. Esto se resolvió matemáticamente imponiendo $E = \max(\lceil \rho_2 Q \rceil, Q) \geq Q$, lo que asegura que cualquier subconjunto recién retirado siga siendo candidato. A nivel de software, generar el vecino sobre *copias* del vector y de w evitó corromper la solución incumbente, y el sembrado por corrida (`random.Random(12345+c)`) garantizó la reproducibilidad exigida por el experimento.

¿Qué modificaciones se proponen para mejorar la convergencia o los tiempos? Se proponen las siguientes líneas de mejora:

- **Evaluación incremental (Δ -evaluación):** en lugar de recalcular z sumando todos los costos seleccionados en cada iteración, mantener z de forma incremental durante AGREGAR/QUITAR reduciría el costo por movimiento.
- **Reinicios y multi-arranque:** dado que la convergencia es muy rápida y estable, un esquema *multi-start* con presupuesto repartido podría explorar regiones distintas del espacio en instancias más difíciles.
- **Enfriamiento adaptativo / recalentamiento:** ajustar dinámicamente T en función de la tasa de aceptación, o recalentar al detectar estancamiento, equilibraría mejor exploración e intensificación.
- **Memoria de corto plazo (tabú):** prohibir temporalmente visitar subconjuntos recién retirados evitaría ciclos.
- **Paralelización de las réplicas:** las 10 corridas son independientes (*embarrassingly parallel*) y pueden ejecutarse en paralelo para reducir el tiempo total de experimentación.
- **Estructuras de datos más rápidas:** representar la cobertura mediante *bitsets* aceleraría las operaciones de conjunto en instancias de gran tamaño.

¿Es viable económica y técnicamente en la realidad chilena? Sí, con matices. El SCP modela numerosos problemas operativos reales en Chile: la **localización de servicios de emergencia** (bomberos, ambulancias, SAMU) para cubrir territorios con el menor costo, la **cobertura de antenas** de telecomunicaciones, la **asignación de turnos y tripulaciones** en transporte (buses RED, aerolíneas) y la **planificación logística** en minería, retail y forestal. Desde el punto de vista económico, la solución es *altamente viable*: el algoritmo corre en segundos, utiliza exclusivamente la biblioteca estándar de Python (sin licencias de *solvers* comerciales como Gurobi o CPLEX, que suelen ser onerosas) y se ejecuta en hardware modesto, lo que lo hace accesible para PyMEs, municipios y el sector público con presupuestos acotados.

El análisis crítico, no obstante, debe reconocer las limitaciones. Las metaheurísticas **no entregan garantías de optimalidad**: para aplicaciones reguladas o críticas donde se requieren cotas demostrables, pueden ser preferibles métodos exactos o híbridos (matheurísticas). Además, las instancias reales suelen ser mayores y más ruidosas que las de prueba, por lo que el ajuste de parámetros (ρ_1 , ρ_2 , T_0 , CF) debería recalibrarse por dominio. En conjunto, para la mayoría de los problemas operativos de cobertura en el contexto chileno, el **equilibrio entre calidad de la solución, bajo costo de implementación y rapidez** hace que técnicas estocásticas como el Recocido Simulado sean una opción técnica y económicamente atractiva.

Referencias

- [1] Jacobs, L. W., & Brusco, M. J. (1995). A local-search heuristic for large set-covering problems. *Naval Research Logistics*, 42(7), 1129–1140.
- [2] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- [3] Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6), 1087–1092.